Gecko: Versatile Text Embeddings Distilled from Large Language Models

Jinhyuk Lee*, Zhuyun Dai*, Xiaoqi Ren*, Blair Chen, Daniel Cer, Jeremy R. Cole, Kai Hui, Michael Boratko, Rajvi Kapadia, Wen Ding, Yi Luan, Sai Meher Karthik Duddu, Gustavo Hernandez Abrego, Weiqiang Shi, Nithi Gupta, Aditya Kusupati, Prateek Jain, Siddhartha Reddy Jonnalagadda, Ming-Wei Chang and Iftekhar Naim
*Equal contributions

arXiv:2403.20327v1 [cs.CL] 29 Mar 2024

We present Gecko, a compact and versatile text embedding model. Gecko achieves strong retrieval performance by leveraging a key idea: distilling knowledge from large language models (LLMs) into a retriever. Our two-step distillation process begins with generating diverse, synthetic paired data using an LLM. Next, we further refine the data quality by retrieving a set of candidate passages for each query, and relabeling the positive and hard negative passages using the same LLM. The effectiveness of our approach is demonstrated by the compactness of the Gecko. On the Massive Text Embedding Benchmark (MTEB), Gecko with 256 embedding dimensions outperforms all existing entries with 768 embedding size. Gecko with 768 embedding dimensions achieves an average score of 66.31, competing with 7x larger models and 5x higher dimensional embeddings.

1. Introduction

Text embedding models represent natural language as dense vectors, positioning semantically similar text near each other within the embedding space (Gao et al., 2021; Le and Mikolov, 2014; Reimers and Gurevych, 2019). These embeddings are commonly used for a wide range of downstream tasks including document retrieval, sentence similarity, classification, and clustering (Muennighoff et al., 2023). Instead of building separate embedding models for each downstream task, recent efforts seek to create a single embedding model supporting many tasks.

The recent development of general-purpose text embedding models presents a challenge: these models require large amounts of training data to comprehensively cover desired domains and skills. Recent embedding efforts have focused on using extensive collections of training examples (Li et al., 2023; Wang et al., 2022). Large language models (LLMs) offer a powerful alternative, as they contain vast knowledge across various domains and are known to be exceptional few-shot learners (Anil et al., 2023; Brown et al., 2020). Recent work demonstrates the effectiveness of using LLMs for synthetic data generation, but the focus has primarily been on augmenting existing human-labeled data or improving performance in specific domains (Dai et al., 2022; Jeronymo et al., 2023). It motivates us to re-examine: to what extent can we leverage LLMs directly to improve text embedding models?

In this work, we present Gecko, a highly versatile yet efficient embedding model, powered by the vast world knowledge of LLMs. Our approach leverages insights from knowledge distillation to create a two-step LLM-powered embedding model. Starting with a large corpus of (unlabeled) passages, we use a few-shot prompted LLM to generate a relevant task and query for each passage, similar to Dai et al. (2022) and Wang et al. (2023). We then embed the concatenated task and query using a pretrained embedding model to obtain nearest neighbor passages, use an LLM to rerank the passages, and obtain positive and negative passages based on the LLM scores. The reranking step is key to enhance the quality as we discover that the best passage to answer the generated query often differs from the original source passage. We show that using our LLM-based dataset, FRet, alone can lead to significantly improvement, setting a strong baseline as a zero-shot embedding model on MTEB.

Corresponding author(s): jinhyuklee@google.com
© 2024 Google DeepMind. All rights reserved

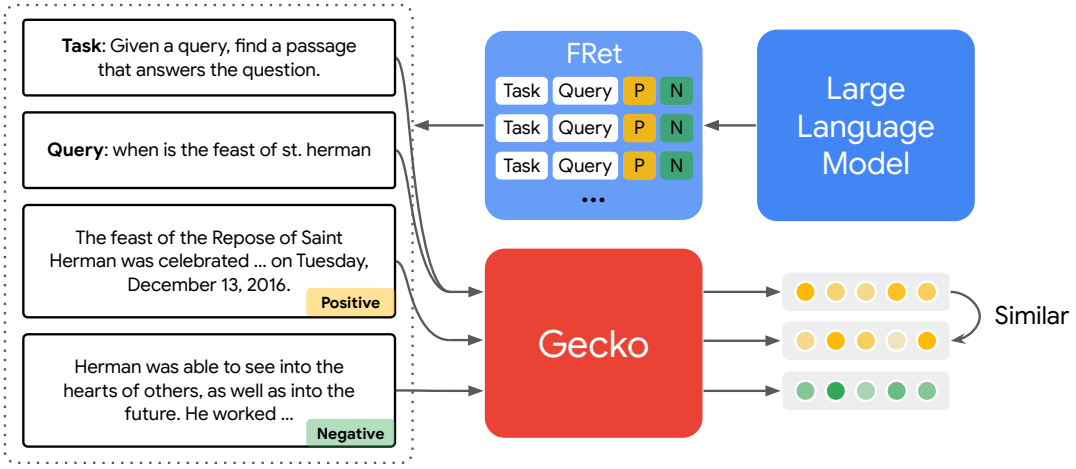

Figure 1 | Overview of Gecko. Gecko is a versatile text embedding model trained on a variety of tasks including document retrieval, semantic similarity, and classification. To train Gecko, we utilize FRet where queries are generated from LLMs, and their positive and negative passages are mined by LLMs.

By combining this LLM-generated and LLM-ranked data with human-annotated data, our model, Gecko-1B with 768-dimensional embeddings, achieves the best performance on the popular MTEB benchmark (Muennighoff et al., 2023) among the models with compatible embedding dimensions and model sizes. Moreover, Gecko often outperforms other systems that use either larger base models (7B) or higher dimensional embeddings (1k to 4k).

2. Related Work

Text Embedding Models Text embeddings convert textual inputs into uniform-sized vectors, supporting downstream tasks such as semantic similarity, information retrieval, clustering, and classification. Recent models, including SBERT (Reimers and Gurevych, 2019), Universal Sentence Encoder (Cer et al., 2018), and Sentence T5 (Ni et al., 2022), attempt to provide general purpose embeddings suitable for various NLP tasks. Despite attempting to be general-purpose, studies indicate that these embedding models struggle to generalize across tasks and domains, motivating the creation of unified models trained across diverse tasks (Asai et al., 2022; Su et al., 2022) and benchmarks such as MTEB (Muennighoff et al., 2023) focused on novel task and domain generalization. Inspired by these prior works, we develop a versatile embedding model by creating the LLM-generated FRet dataset from a large and diverse corpus encompassing a wide variety of task types.

Contrastive Learning One of the critical components of contrastive learning is to find proper negative examples for a query (Gao et al., 2021; Karpukhin et al., 2020; Lee et al., 2021). For example, Xiong et al. (2020) proposed to select hard negatives from a large corpus using an asynchronously-updated approximate nearest neighbor index. Other previous work has denoised the hard negatives based on confidence scores (Qu et al., 2021; Ren et al., 2021) or distilled knowledge from cross-attention rerankers into the dual-encoders (Izacard and Grave, 2021; Sachan et al., 2023; Santhanam et al., 2022). In our work, using LLMs, we study the effect of mining better positive examples for a query while finding useful hard negatives as well. While similar in spirit to previous distillation approaches, using this hard selection of positive and negative passages aligns well with the format of existing human-annotated training data, allowing us to train on both.

Synthetic Data Generation When applying text embedding models to new tasks and domains, we often want to have relevant queries and labels for these target domains, but they are often unavailable or prohibitively expensive to collect. To address this issue, several works (Bonifacio et al., 2022; Dai et al., 2022; Jeronymo et al., 2023; Khramtsova et al., 2024) propose a few-shot prompted query generation approach. They generate synthetic queries by few-shot prompting LLMs to create a domain-specific training dataset, which has been shown to be very successful on the zero-shot information retrieval benchmark (Thakur et al., 2021). In contrast to generating domain-specific queries for domain adaptation, our work aims to distill more general-purpose knowledge of LLMs into a text embedding model, resulting in a versatile text embedding model that achieves strong performance on MTEB (Muennighoff et al., 2023).

Retrieval with Instructions Previously, Dai et al. (2022) demonstrated that there exist different intents for different retrieval tasks. For instance, given a search query, users might want to find a similar query, or they might want to read a passage that directly answers the query. Recent work has explored implementing a retriever that changes the retrieval behavior for different intents. Asai et al. (2022) and Su et al. (2022) introduce "retrieval with instructions," where a dense retriever is trained to follow an instruction that was given along with the query. Wang et al. (2023) also explores how LLMs can generate synthetic task instructions and associated queries, but for more general-purpose text embeddings similar to ours. They use a two-step prompt to encourage the diversity of the synthetic data: first prompting an LLM to come up with a task and then generating an example (query, positive passage, and negative passage) based on the task. In our work, we also synthesize task-query pairs to increase the diversity of the synthetic data. Unlike Wang et al. (2023), however, we generate synthetic task and query pairs from the web passages, basing our FRet dataset on real user-facing content. We also use LLMs to decide which web passages can be used as positive or negative targets for each generated query.

3. Training Recipe for Gecko

Gecko is based on a 1.2B parameter pre-trained transformer language model that undergoes two additional training stages: pre-finetuning and fine-tuning. First, we extend the pre-finetuning recipe from previous work (Ni et al., 2021; §3.1). For fine-tuning, our main contribution is to create a novel fine-tuning dataset for a diverse set of downstream tasks via a two-step LLM distillation, which identifies both positive and hard negative passages for each generated query (§3.2). We coin this dataset as FRet, the **F**ew-shot Prompted **Ret**rieval dataset. For the fine-tuning mixture, FRet is combined with a diverse set of academic datasets formatted in a similar way: each with a task description, input query, positive passage, and negative passage (§3.3).

3.1. Pre-finetuning

Following the prior work (Neelakantan et al., 2022; Ni et al., 2021; Wang et al., 2022), our pre-finetuning procedure relies on self-supervised tasks over a large text corpus as described below.

Training Mixture We use two pre-finetuning datasets. First, we use the large-scale community QA dataset by Ni et al. (2021), which includes text pairs such as question-answer pairs from online forums and QA websites. Next, we crawl a corpus of title-body text pairs from the Web, which can be found from almost every website as naturally occurring pairs. Despite its simplicity, Wang et al. (2022) showed that these naturally occurring text pairs are useful for pre-finetuning embedding models.

Training Objective Pre-finetuning on a large amount of unsupervised text pairs has been shown to improve performance for smaller-scale dual encoders for various downstream tasks including document retrieval (Izacard et al., 2022; Lee et al., 2019) and semantic similarity (Gao et al., 2021). The goal of the pre-finetuning stage is to expose the model to a large amount of textual diversity, which seems necessary for the compact text embedding models that we aim to train.

We begin with a pre-trained language model $\mathcal{M}$ where $\mathcal{M}$ outputs a series of contextualized token embeddings $\mathbf{W} \in \mathbb{R}^{n \times d}$ given a sequence of n tokens and an embedding dimension of d. Given a set of text pairs $\mathcal{D}_{\text{pre}} = \{(q_i, p_i)\}_{i=1}^{N}$ for pre-finetuning, we obtain the vector representations of q_i and p_i by taking the mean of $\mathbf{W}$ along the n axis. We first prepend a dataset-specific task feature t before each query, so each query is informed of which task is being optimized.

$$\begin{aligned} \mathbf{q}_i &= \texttt{mean_pool}_{|t|+|q_i|}\left[\mathcal{M}(t \oplus q_i) \in \mathbb{R}^{(|t|+|q_i|)\times d}\right] \in \mathbb{R}^d \\ \mathbf{p}_i &= \texttt{mean_pool}_{|p_i|}\left[\mathcal{M}(p_i) \in \mathbb{R}^{|p_i|\times d}\right] \in \mathbb{R}^d. \end{aligned} \tag{1}$$

For pre-finetuning, we use simple task features such as *question answering* or *search result* for t depending on the dataset. Then, for each mini-batch of size B, we optimize the contrastive learning objective with in-batch negatives:

$$\mathcal{L}_{\text{pre}} = \frac{1}{B}\sum_{i=1}^{B}\left[-\log \frac{e^{\text{sim}(\mathbf{q}_i,\mathbf{p}_i)/\tau}}{\sum_{j=1}^{B} e^{\text{sim}(\mathbf{q}_i,\mathbf{p}_j)/\tau}}\right]. \tag{2}$$

In this work, we use the cosine similarity for the similarity function, $\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{||\mathbf{x}|| \cdot ||\mathbf{y}||}$, with a temperature parameter τ. Note that we do not utilize hard negatives during pre-finetuning and utilize the maximum batch size that fits into the device. This has been found to be effective for document retrieval tasks as observed in previous work (Li et al., 2023; Wang et al., 2022).

3.2. FRet: Two-Step LLM Distillation

In this section, we introduce our two-stage approach that uses LLMs to generate FRet. Traditional approaches for training embedding models often rely on large, manually labeled datasets. However, creating such datasets is time-consuming, expensive, and often results in undesirable biases and lack of diversity. In this work, we present a novel method for generating synthetic data for training multi-task text embedding models, leveraging the power of LLMs through a two-step distillation process. The overall process of generating FRet is illustrated in Figure 2.

LLM-based Diverse Query Generation One of the challenges of using manually crafted queries is to ensure that the queries cover a diverse set of tasks and linguistic patterns. With LLMs, these variables are relatively easy to control as we can design the prompt to specify the diversity. In this work, we employ few-shot prompts to control the diversity of queries. Our LLM is instructed to read a sampled web passage and generate both the task description and a relevant query for the task:

$$\text{LLM}(\mathbb{P}_{\text{QG}}, p_{\text{seed}}) \rightarrow (t, q)$$

where p_{seed} is a passage drawn randomly from the web corpus C and $\mathbb{P}_{\text{QG}}$ is a fixed prompt. The prompt, $\mathbb{P}_{\text{QG}}$, is identical for every example and consists of few-shot examples and instructions. The LLM generates a task description t, which describes the type of retrieval—for example, *'Given a query, find a passage that has the answer to the query'* (question answering) or *'Given a query, find a passage that allows you to check whether the query is true or not'* (fact checking)—and also a query q that aligns

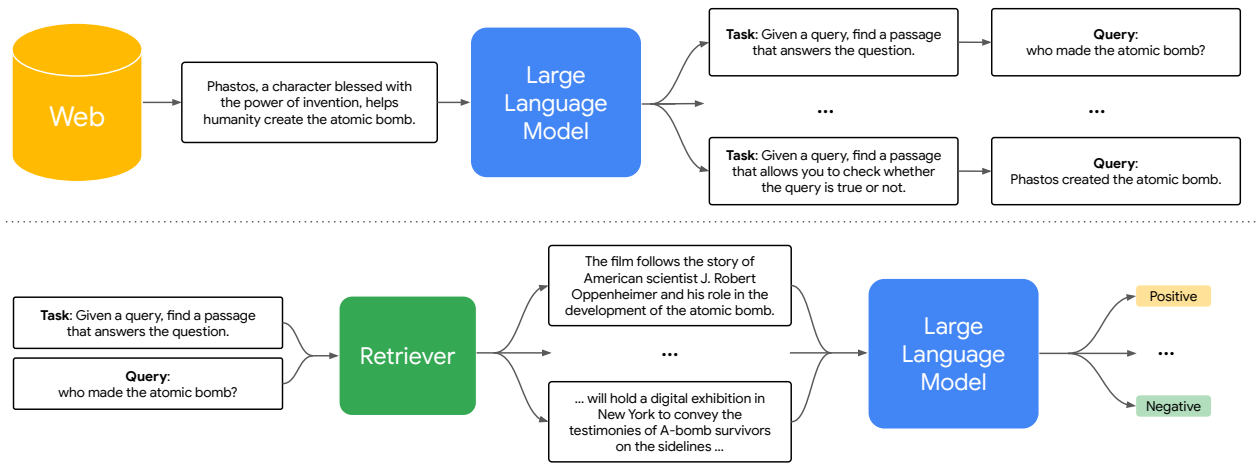

Figure 2 | Overview of FRet. Given a sampled passage from the web, FRet first utilizes LLMs to generate a relevant task and a query for the passage (top). Then, each query and task is fed into a pre-trained embedding model to obtain nearest neighbor passages, which are then scored by the LLM to mine positive and negative passages (bottom). Note that the original web passage does not necessarily become a positive passage as LLMs can find a more relevant passage as shown above.

with the task. By sampling over such free-form task descriptions, we guide the LLM to produce a wide range of queries. These pairs are later used to train our embedding models, teaching the models to associate a query and its corresponding instructions with the target passage.

The diversity of FRet comes from two sources. First, a web corpus inherently contains a variety of topics as well as styles of writing, such as blog posts, news, Wikipedia-like content, and forum posts. Second, by adding many diverse task descriptions in the prompt, we encourage the LLM to generate more diverse task descriptions and therefore more diverse queries. Similar to Dai et al. (2022), our method can be applied to any corpus of passages. Our method is different from approaches such as Wang et al. (2023), where LLMs generate both synthetic queries and synthetic passages.

LLM-based Positive and Negative Mining Most models that utilize synthetic queries are trained with (q, p_{seed}) pairs, which assumes that p_{seed} is a good positive target for q (Dai et al., 2022; Jeronymo et al., 2023). While this is likely true in most cases, we hypothesize that there could be a more relevant passage than p_{seed} somewhere in our corpus of web passages. Essentially, in the previous section, we sampled $\mathrm{P}(t, q \mid p_{\text{seed}})$ from the LLM, but this does not guarantee that p_{seed} maximizes $\mathrm{P}(p \mid q, t)$ over all the passages in the corpus. This intuition is supported by our observation that generated queries often focus on a particular aspect of a relatively long passage. Hence, we propose a method that leverages LLMs to discover more relevant positive passages along with a good hard negative for the generated query.

In particular, we use an existing embedding model[1] to retrieve top N neighbors $P = \{p^{(1)}, \ldots, p^{(N)}\}$ from the corpus given a generated query q. We then employ the same LLM used for the query generation to rank these retrieved passages based on their relevance to the query. Specifically, we use two well-known few-shot prompted LLM ranking functions: query likelihood and relevance classification. Query likelihood uses an LLM to measure the log-likelihood of a generated query q given a passage p, i.e., $\mathrm{QL}(q, p) = \mathrm{LLM}(q \mid p, \mathbb{P}_{\mathrm{QL}})$ (Sachan et al., 2022). Herein, $\mathbb{P}_{\mathrm{QL}}$ is a prompt containing an instruction for judging query likelihood and several few-shot examples of relevant

[1] In this work, we train an initial embedding model with (q, p_{seed}) pairs, treating in-batch passages as random negatives.

query and passage pairs (Drozdov et al., 2023). Relevance classification (Zhuang et al., 2023) uses an LLM to measure the log-likelihood of a specific relevance label given the query q and a passage p, i.e., $\text{RC}(q, p) = \text{LLM}(\text{label} \mid q, p, \mathbb{P}_{\text{RC}})$, where $\mathbb{P}_{\text{RC}}$ is a prompt with few-shot examples for grading the relevance of each query-passage pair. The prompts $\mathbb{P}_{\text{QL}}$ and $\mathbb{P}_{\text{RC}}$ are identical for every example. Our pilot study demonstrated that each prompting method (i.e. QL and RC) excels in different tasks, so we ensemble the rankings from two different prompting results with the standard Reciprocal Rank Fusion (RRF) approach (Cormack et al., 2009), obtaining a ranking function $R(q, p)$. As shown in Appendix A, the ensembling greatly improves the robustness of our model across diverse tasks.

Given the scores from LLMs after ensembling, we index the set of passages P according to their ranking, i.e. $P = \{p_1, \ldots, p_N\}$ where if $i < j$, $R(q, p_i) \geq R(q, p_j)$. We then choose a new positive target:

$$p^+ = \arg\max_{p \in P} R(q, p) = p_1$$

Importantly, p^+ can be different from p_{seed} and conveys an approximation to the global preference of the LLM over the entire corpus. Table 5 lists examples where the p^+ differs from p_{seed}, demonstrating that the pair (q, p_{seed}) can be sub-optimal and there can be more relevant passages for q globally. We find that the relabeling of the positive passage (i.e., $p^+ \neq p_{\text{seed}}$) happens for about 15% in our dataset.

Similarly, the LLM scores can also be used to select hard negative passages. One straightforward option is to select the lowest scoring negative, i.e. $p^- = p_N$. Another is to sample from the remaining nearest neighbors, i.e. $p^- \sim P \setminus \{p^+\}$. We explore both options in §4.3. Combining all of our generation results along with the positive and negative mining, we create the FRet dataset, comprised of 6.6M examples, each containing a task, a query, a positive passage, and a negative passage.

3.3. Unified Fine-tuning Mixture

We combine FRet with other academic training datasets in the same format: task description, input query, positive passage (or target), and negative passage (or distractor), creating a novel fine-tuning mixture. We then train our embedding model, Gecko, using this mixture with a standard loss function.

Academic Data In addition to FRet, we use the following academic training datasets: Natural Questions (Kwiatkowski et al., 2019), HotpotQA (Yang et al., 2018), FEVER (Thorne et al., 2018), MedMCQA (Pal et al., 2022), SNLI (Bowman et al., 2015), MNLI (Williams et al., 2018), and several classification datasets from Huggingface. For the multilingual model, we add training sets from MIRACL (Zhang et al., 2023). All datasets are pre-processed to have a unified encoding format (Appendix B), containing a task description, a query, a positive passage, and a negative passage.

Classification Data for Contrastive Learning We aim to seamlessly incorporate the classification training sets into our contrastive learning objective without any performance degradation on other tasks such as document retrieval. Specifically, given a classification input text x with a label $y \in \mathcal{Y}$, we pair each input x with another input x^+, which shares the same label y and then use x^+ as a positive target for x. At the same time, we randomly select a hard negative input x^- which has any label other than y. This approach is a simple version of the classification datasets pre-processed by Su et al. (2022) but avoids using any model-specific embeddings. During our experiments, we found that each x^+ might overlap with other positive examples within the mini-batch, creating a false negative problem among the in-batch negatives. Hence, we assign a unique ID to each triple (x, x^+, x^-) and append the same unique ID to x, x^+, and x^-. This effectively makes the in-batch negatives trivial for the model to distinguish them, because if the unique ID does not match, then it is never the correct answer. Thus, the model focuses on differentiating x^+ and x^- given x.

	Dim.	# Params.	Class.	Cluster.	Pair.	Rerank.	Retrieval	STS	Summary	Avg.
gritlm-8x7b	4,096	56B	78.53	50.14	84.97	59.80	55.09	83.26	29.82	65.66
e5-mistral-7b-instruct	4,096	7B	78.47	50.26	**88.34**	60.21	56.89	**84.63**	**31.40**	66.63
echo-mistral-7b-instruct	4,096	7B	77.43	46.32	87.34	58.14	55.52	82.56	30.73	64.69
gritlm-7b	4,096	7B	**79.46**	**50.61**	87.16	**60.49**	**57.41**	83.35	30.37	**66.76**
text-embedding-3-large (OpenAI)	3,072	n/a	75.45	49.01	85.72	59.16	55.44	81.73	29.92	64.59
gtr-t5-xxl	768	5B	67.41	42.42	86.12	56.66	48.48	78.38	30.64	58.97
gtr-t5-xl	768	1.2B	67.11	41.51	86.13	55.97	47.96	77.80	30.21	58.42
instructor-xl	768	1.5B	73.12	44.74	86.62	57.29	49.26	83.06	32.32	61.79
text-embedding-3-large-256 (OpenAI)	256	n/a	71.97	46.23	84.22	57.99	51.66	81.04	29.92	62.00
gecko-1b-256	256	1.2B	78.99	45.07	87.25	57.78	52.44	84.93	32.36	64.37
gecko-1b-768	768	1.2B	**81.17**	**47.48**	**87.61**	**58.91**	**55.70**	**85.06**	**32.63**	**66.31**
– zero-shot (FRet-only)	768	1.2B	70.26	46.82	86.27	57.60	53.16	83.14	32.16	62.64

Table 1 | Results on MTEB. We categorize models into two groups based on their embedding dimension (Dim.) and the number of parameters (# Params.). We report the average performance on seven different tasks: Classification (Class.), Clustering (Cluter.), Pair Classification (Pair.), Reranking (Rerank.), Retrieval, STS, and Summary. The last column shows the average performance across all 56 datasets from the seven tasks. In the last row, we show the performance of a zero-shot Gecko model, solely trained on FRet without any human-labeled data or MTEB in-domain training datasets. Please refer to Appendix C for the result and the instruction per dataset.

Training Objective For fine-tuning, we are given a set of M fine-tuning datasets (including FRet) that are comprised of a query-specific task description, an input, a positive target, and a hard negative: $[\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(M)}]$ where $\mathcal{D}^{(m)} = \{(t_i, q_i, p_i^+, p_i^-)\}_{i=1}^N$. We obtain the vector representations $\mathbf{q}_i$, $\mathbf{p}_i^+$, and $\mathbf{p}_i^-$ similar to eq. (1) where t_i is used for the input: $\mathbf{q}_i = \texttt{mean_pool}[\mathcal{M}(t_i \oplus q_i)]$.

For fine-tuning we optimize the in-batch cross-entropy loss, where query q_i should distinguish p_i^+ from the hard negative p_i^-, other passages in the batch $\{p_j^+\}_{j=1}^B$, and other queries in the batch $\{q_j\}_{j=1}^B \setminus \{q_i\}$. The use of other queries in the batch is also known as "same-tower negatives" (Moiseev et al., 2023). Given a mini-batch of size B, we optimize the following objective:

$$\mathcal{L}_{\text{main}} = \frac{1}{B}\sum_{i=1}^{B}\left[-\log \frac{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)/\tau}}{\sum_{j=1}^{B}\left(e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_j^+)/\tau} + \mathbb{1}_{[j\neq i]} e^{\text{sim}(\mathbf{q}_i, \mathbf{q}_j)/\tau}\right) + e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^-)/\tau}}\right]. \tag{3}$$

For the same-tower negatives, we used the indicator variable $\mathbb{1}_{[j\neq i]}$ to denote that we are iterating over j except for the current target index i. Intuitively, same-tower negatives are helpful for symmetric text embedding tasks such as measuring the semantic similarity of two sentences, because $\{\mathbf{q}_j\}_{j=1}^B$ shares the same modality with $\mathbf{q}_i$: in this case, both are queries. Finally, to support multiple different dimensions of embeddings with a single model, we add the MRL loss (Kusupati et al., 2022), which optimizes eq. (3) with sub-dimensions smaller than d. In our experiments, we use two embedding dimensions $d = 768$ and $d = 256$ for Gecko.

4. Experiments

We mainly evaluate Gecko on the Massive Text Embedding Benchmark (MTEB), which contains 56 datasets on retrieval, semantic textual similarity (STS), clustering, classification, pair classification, reranking, and summarization. We analyze how each component of Gecko and FRet contribute to the performance, providing insights on building heterogeneous text embedding models.

	MIRACL (Avg.)
Per-language models	
BM25	38.5
mDPR	41.8
BM25 + mDPR (hybrid)	56.6
One model for all languages	
mDPR (en)	39.7
mContriever (en)	37.8
mContriever	52.7
SWIM-X	46.4
mContriever-X	55.4
text-embedding-3-large (OpenAI)	54.9
gecko-multilingual-1b	**56.2**

Table 2 | Results on MIRACL. We report average nDCG@10 on multilingual retrieval tasks in 18 languages (ar, bn, en, es, fa, fi, fr, hi, id, ja, ko, ru, sw, te, th, zh, de, yo). Each row shows the performance of a single multilingual retriever.

Positive (p^+)	Hard Negative (p^-)	BEIR	STS
MS-MARCO			
p_{seed}	None	49.87	**79.38**
p_{seed}	$p \sim P \setminus \{p_{\text{seed}}\}$	50.31	78.17
p_1	$p \sim P \setminus \{p_1\}$	52.03	78.96
p_1	p_{20}	**52.29**	78.96
FRet			
p_{seed}	None	52.33	82.66
p_{seed}	$p \sim P \setminus \{p_{\text{seed}}\}$	51.37	82.00
p_{seed}	p_{20}	51.96	82.26
p_1	None	53.07	82.88
p_1	$p \sim P \setminus \{p_1\}$	52.60	82.85
p_1	p_{20}	**53.39**	**83.14**

Table 3 | With MS-MARCO and FRet, we test different strategies of choosing positive and hard negative passages. We train each model and report its performance on BEIR (nDCG@10) and STS (Spearman Correlation) performance.

4.1. Main Results

Table 1 summarizes the performance of Gecko and other baselines on MTEB. For baselines, we report the performance of text embedding models whose recipes are fully (or partly) available. Gecko significantly surpasses all similarly-sized baselines (<= 1k embedding dimensions, <= 5B parameters) on every text embedding task in the MTEB benchmark. Gecko-1b-256 demonstrates superior quality compared to text-embedding-3-large-256 (OpenAI; Neelakantan et al. 2022), GTR (Ni et al., 2021), and Instructor (Su et al., 2022). Gecko-1b-768 often matches or exceeds the performance of even larger models, including text-embedding-3-large (OpenAI), E5-mistral (Wang et al., 2023), GRit (Muennighoff et al., 2024), and Echo embeddings (Springer et al., 2024). Notably, these models all use 3-4k dimensional embeddings and exceed 7B parameters. We observe that Gecko is particularly good at balancing retrieval and STS performance, and sets a new state-of-the-art on classification, STS, and summary. Surprisingly, the performance of Gecko trained solely on FRet, which makes MTEB a pure zero-shot benchmark, shows strong performance compared to other baselines.

4.2. Multilingual Retrieval Results

Table 2 summarizes the performance of Gecko and other baselines on MTEB. We train a multilingual version of Gecko with multilingual language models (Team et al., 2023; Xue et al., 2021) with the same recipe as Gecko, but add the MIRACL training dataset in the mixture. Note that FRet is provided only in English and the main difference of gecko-multilingual-1b with others is the use of FRet in its training set. We find that while we only generated English-only dataset from LLMs, this translates well to other multilingual tasks achieving superior performance compared to others.

4.3. Analysis

LLM as a Labeler In Table 3, we test different labeling strategies for FRet where we use different positive and hard negative passages. For positive passages, we try 1) the original passage where the queries were generated (i.e. p_{seed}), or 2) the top-1 passage selected by an LLM out of the nearest neighbor passages (including the original one) of a generated query (i.e. p_1). For negative

	Class.	Cluster.	Pair.	Rerank.	Retrieval	STS	Summary	Avg.
Baseline (Ni et al., 2021)	67.11	41.51	86.13	55.97	47.96	77.80	30.21	58.42
FRet synthetic data ablation								
FRet-question-answering	69.39	45.58	**84.40**	56.30	49.65	78.98	31.17	60.32
FRet-search-result	70.41	44.12	82.99	56.50	49.65	78.82	31.27	60.17
FRet-fact-checking	**70.81**	**45.70**	81.63	**57.31**	49.38	79.34	30.99	60.56
FRet-sentence-similarity	70.25	45.60	81.46	56.73	47.26	**82.02**	**31.80**	60.30
FRet-all-tasks (300K)	70.25	44.56	85.37	56.46	**50.19**	80.07	30.67	60.70
[+] Uniform task sampling	70.57	45.00	85.35	56.84	49.67	80.70	31.34	**60.87**
[−] Unified format	61.72	45.58	82.89	54.52	45.82	79.06	30.29	57.45
Human data ablation								
FRet (6.6M)	70.26	46.82	86.27	57.60	53.16	83.14	32.16	62.64
[+] NLI datasets	71.86	46.91	86.60	57.51	52.93	84.74	32.11	63.24
[+] Class. datasets	81.00	46.85	86.13	57.80	52.84	82.78	32.35	64.82
[+] Full mixture	**81.17**	**47.48**	**87.61**	**58.91**	**55.70**	**85.06**	**32.63**	**66.31**

Table 4 | Does the diversity of FRet matter when training versatile embedding models? We test different subsets of FRet for training and report their performance on MTEB. From the four most frequent tasks in FRet (e.g., FRet-question-answering), we sample 300k training examples. For FRet-all-tasks, we sample 75k training examples from each task to form 300k training examples. We also test sampling FRet examples uniformly across different tasks and replacing the unified format (Appendix B) with naive concatenation of tasks and text. In the bottom rows, we show the performance of using all FRet training data along with human annotated NLI and classification datasets.

passages, we try 1) a random nearest neighbor passage that is different from the original passage (i.e. $p \sim P \setminus \{p_{\text{seed}}\}$), or 2) the k-th passage as ranked by the LLM out of the nearest neighbor passages (including the original one) for the given query (i.e. p_k). From the result, we find that using the most relevant passage chosen by an LLM is always better than using the original passage as positive. This implies that the original passage is not necessarily best passage to use as a positive target despite the fact that the query was generated from it. In our qualitative analysis in Table 5, we show that such cases happen quite often.

Diversity of FRet FRet provides queries in multiple tasks including question answering, search result, fact checking, and sentence similarity. In Table 4, we test how the diversity of FRet influences model generalizability across tasks in MTEB. First, we train individual models each using 300k data from a specific task (e.g., FRet-question-answering). Additionally, we train models on 300k samples drawn across all four tasks (75k per task; FRet-all-tasks) with original sampling distribution or uniform sampling distribution. We observe superior performance from the FRet-all-tasks model, particularly when tasks were uniformly sampled. We also find that the unified formatting (Appendix B) affects the quality of embeddings significantly, as it helps the model better separate different tasks.

Learning Semantic Similarity and Classification In the last rows of Table 4, we show how Gecko learns better semantic similarity and classification. We use the symmetric format (Sym.) as well as the same tower negatives for learning better semantic similarity. Along with the NLI datasets, it drastically improves the STS performance by 1.6 on average. Our strategy of combining classification datasets also improve the performance on classification by a large margin without significant performance degradation on other tasks. Using the full FRet mixture gives us the final performance of 66.31.

Seed Passage (p_{seed})	Recently, Marvel's The Eternals has become the topic of a great deal of online discourse, in part because of a scene where Phastos, a character blessed with the power of invention, helps humanity create the atomic bomb. As you can probably imagine, Twitter saw this and lost it.
Generated Task (t)	*Given a query, find a passage that has the answer to the query.*
Generated Query (q)	*who made the atomic bomb?*
LLM-mined Positive (p_1)	The film follows the story of American scientist J. Robert Oppenheimer and his role in the development of the atomic bomb.
LLM-mined Negative (p_{20})	Amid deepening crises around the world with nuclear undertones, a research team from the University of Tokyo will hold a digital exhibition in New York to convey the testimonies of A-bomb survivors on the sidelines of the United Nations review conference of a nuclear nonproliferation treaty.
Seed Passage (p_{seed})	moose - online shopping for canadians. The 2010 Vancouver Winter Olympics $75 gold coins were sold individually or in sets of three coins. The three different sets offered were Canadian Wildlife, Canadian Emblems and Vancouver 2010 Olympic Winter Games.
Generated Task (t)	*Given a query, find a passage that might show up as a search result.*
Generated Query (q)	*2010 olympic winter games*
LLM-mined Positive (p_1)	The 2010 Winter Olympics return to North America on February 12th, when the world of snow sport enthusiasts descend upon one of North America's most beautiful cities, Vancouver.
LLM-mined Negative (p_{20})	Published: 9:42pm, 12 Feb, 2018 High winds caused havoc at the Pyeongchang Winter Games on Monday as Olympics chief Thomas Bach dismissed concerns North Korea had tried to "hijack" the competition for political gain.
Seed Passage (p_{seed})	Tagged: Batman, Robin, DC, DC Comics, Comics, ...
Generated Task (t)	*Given a query, find a passage that allows you to check whether the query is true or not.*
Generated Query (q)	*Batman is from DC comics*
LLM-mined Positive (p_1)	The Batman is an American superhero film based on the DC Comics character of the same name. Produced by DC Films and distributed by Warner Bros. Pictures, it is a reboot of the Batman film franchise.
LLM-mined Negative (p_{20})	"One of my employees wants to dress up in Batman attire," Gaskins said. "As long as he's at work, I told him it was fine." New York Times News Service contributed to this report.

Table 5 | Examples for LLM-mined positives and negatives. While the intent of each query aligns with each task, LLM-mined positive is often more relevant than the seed passage for the generated query.

Qualitative Analysis Table 5 showcases the advantages of LLM relabeling. We provide examples of the original seed passage, generated task and query, and the LLM-mined positive and negative passages. First, we observe that the LLM does generate diverse tasks and queries by conditioning on seed passages p_{seed}. Second, the table highlights the LLM's ability to find a passage (p_1) that provides a more direct and relevant answer to the generated query than the seed passage (p_{seed}). Furthermore, LLM-ranked hard negatives make a challenging task of understanding nuanced differences. These examples demonstrate how the 2-step LLM distillation process effectively brings the LLM's diverse domain knowledge and global ranking preferences into the text embedding model.

5. Conclusion

In this paper, we introduced Gecko, a versatile text embedding model distilled from large language models. Gecko is trained on an LLM-generated synthetic dataset FRet that contains LLM-ranked positives and negatives. We demonstrate that LLMs can be used to identify better positive as well as negative targets for synthesized queries. We also show how combining this synthetically-generated data in a unified format can lead us to achieve great performance on multiple different tasks at the same time. Our ablation study reveals the importance of LLM-based relabeling and the diversity of the datasets while demonstrating the strong zero-shot generalizability of Gecko.

References

R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

A. Asai, T. Schick, P. Lewis, X. Chen, G. Izacard, S. Riedel, H. Hajishirzi, and W.-t. Yih. Task-aware retrieval with instructions. *arXiv preprint arXiv:2211.09260*, 2022.

L. Bonifacio, H. Abonizio, M. Fadaee, and R. Nogueira. Inpars: Data augmentation for information retrieval using large language models. *arXiv preprint arXiv:2202.05144*, 2022.

S. Bowman, G. Angeli, C. Potts, and C. D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 632–642, 2015.

T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. S. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, et al. Universal sentence encoder for english. In *Proceedings of the 2018 conference on empirical methods in natural language processing: system demonstrations*, pages 169–174, 2018.

G. V. Cormack, C. L. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.

Z. Dai, V. Y. Zhao, J. Ma, Y. Luan, J. Ni, J. Lu, A. Bakalov, K. Guu, K. B. Hall, and M.-W. Chang. Promptagator: Few-shot dense retrieval from 8 examples. *arXiv preprint arXiv:2209.11755*, 2022.

A. Drozdov, H. Zhuang, Z. Dai, Z. Qin, R. Rahimi, X. Wang, D. Alon, M. Iyyer, A. McCallum, D. Metzler, and K. Hui. PaRaDe: Passage ranking using demonstrations with LLMs. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14242–14252, Singapore, Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.950. URL https://aclanthology.org/2023.findings-emnlp.950.

T. Gao, X. Yao, and D. Chen. Simcse: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910, 2021.

G. Izacard and E. Grave. Distilling knowledge from reader to retriever for question answering. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=NTEz-6wysdb.

G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.

V. Jeronymo, L. Bonifacio, H. Abonizio, M. Fadaee, R. Lotufo, J. Zavrel, and R. Nogueira. Inpars-v2: Large language models as efficient dataset generators for information retrieval. *arXiv preprint arXiv:2301.01820*, 2023.

V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Y. Wu, S. Edunov, D. Chen, and W. tau Yih. Dense passage retrieval for open-domain question answering. *ArXiv*, abs/2004.04906, 2020.

E. Khramtsova, S. Zhuang, M. Baktashmotlagh, and G. Zuccon. Leveraging llms for unsupervised dense retriever ranking. *arXiv preprint arXiv:2402.04853*, 2024.

A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, et al. Matryoshka representation learning. *Advances in Neural Information Processing Systems*, 35:30233–30249, 2022.

T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Q. Le and T. Mikolov. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196. PMLR, 2014.

J. Lee, M. Sung, J. Kang, and D. Chen. Learning dense representations of phrases at scale. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6634–6647, 2021.

K. Lee, M.-W. Chang, and K. Toutanova. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6086–6096, Florence, Italy, July 2019. Association for Computational Linguistics.

Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.

X. Ma, L. Wang, N. Yang, F. Wei, and J. Lin. Fine-tuning llama for multi-stage text retrieval. *arXiv preprint arXiv:2310.08319*, 2023.

F. Moiseev, G. H. Abrego, P. Dornbach, I. Zitouni, E. Alfonseca, and Z. Dong. Samtone: Improving contrastive loss for dual encoder retrieval models with same tower negatives. *arXiv preprint arXiv:2306.02516*, 2023.

N. Muennighoff, N. Tazi, L. Magne, and N. Reimers. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2006–2029, 2023.

N. Muennighoff, H. Su, L. Wang, N. Yang, F. Wei, T. Yu, A. Singh, and D. Kiela. Generative representational instruction tuning. *arXiv preprint arXiv:2402.09906*, 2024.

A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy, et al. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005*, 2022.

J. Ni, C. Qu, J. Lu, Z. Dai, G. H. 'Abrego, J. Ma, V. Zhao, Y. Luan, K. B. Hall, M.-W. Chang, and Y. Yang. Large dual encoders are generalizable retrievers. In *Conference on Empirical Methods in Natural Language Processing*, 2021.

J. Ni, G. H. Abrego, N. Constant, J. Ma, K. Hall, D. Cer, and Y. Yang. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1864–1874, 2022.

A. Pal, L. K. Umapathi, and M. Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on health, inference, and learning*, pages 248–260. PMLR, 2022.

Y. Qu, Y. Ding, J. Liu, K. Liu, R. Ren, W. X. Zhao, D. Dong, H. Wu, and H. Wang. Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5835–5847, 2021.

N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

R. Ren, Y. Qu, J. Liu, W. X. Zhao, Q. She, H. Wu, H. Wang, and J.-R. Wen. RocketQAv2: A joint training method for dense passage retrieval and passage re-ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 2825–2835, Online and Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics.

D. Sachan, M. Lewis, M. Joshi, A. Aghajanyan, W.-t. Yih, J. Pineau, and L. Zettlemoyer. Improving passage retrieval with zero-shot question generation. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3781–3797, Abu Dhabi, United Arab Emirates, Dec. 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.249. URL https://aclanthology.org/2022.emnlp-main.249.

D. S. Sachan, M. Lewis, D. Yogatama, L. Zettlemoyer, J. Pineau, and M. Zaheer. Questions are all you need to train a dense passage retriever. *Transactions of the Association for Computational Linguistics*, 11:600–616, 2023.

K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, 2022.

J. M. Springer, S. Kotha, D. Fried, G. Neubig, and A. Raghunathan. Repetition improves language model embeddings. *arXiv preprint arXiv:2402.15449*, 2024.

H. Su, W. Shi, J. Kasai, Y. Wang, Y. Hu, M. Ostendorf, W.-t. Yih, N. A. Smith, L. Zettlemoyer, and T. Yu. One embedder, any task: Instruction-finetuned text embeddings. *arXiv preprint arXiv:2212.09741*, 2022.

G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal. Fever: a large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 809–819, 2018.

L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei. Improving text embeddings with large language models. *arXiv preprint arXiv:2401.00368*, 2023.

A. Williams, N. Nangia, and S. R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of NAACL-HLT*, pages 1112–1122, 2018.

L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808*, 2020.

L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel. mt5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, 2021.

Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin. Miracl: A multilingual retrieval dataset covering 18 diverse languages. *Transactions of the Association for Computational Linguistics*, 11:1114–1131, 2023.

H. Zhuang, Z. Qin, K. Hui, J. Wu, L. Yan, X. Wang, and M. Berdersky. Beyond yes and no: Improving zero-shot llm rankers via scoring fine-grained relevance labels. *arXiv preprint arXiv:2310.14122*, 2023.

Author Contributions

Jinhyuk Lee: Co-lead of FRet and Gecko. Coordinated the project, implemented the main functionality of FRet and Gecko, and led the paper writing. **Zhuyun Dai**: Co-lead of FRet. Implemented the main functionality of FRet and led the paper writing. **Xiaoqi Ren**: Co-lead of Gecko. Implemented the main functionality of Gecko and its multilingual version. **Blair Chen**: Contributed to the MTEB evaluation and ablation study of Gecko. **Daniel Cer**: Contributed to the MTEB evaluation of Gecko and the classification datasets used for Gecko. **Jeremy R. Cole**: Contributed to experiments for generating and filtering FRet and paper writing. **Kai Hui**: Contributed to the use of LLM as a labeler, rank fusion, and paper writing. **Michael Boratko**: Contributed to the project coordination and paper writing. **Rajvi Kapadia**: Contributed to the use of LLM for the distillation. **Wen Ding**: Contributed to the hyperparameter tuning and ablation study of Gecko. **Yi Luan**: Contributed to the use of LLM as a labeler and paper writing. **Sai Meher Karthik Duddu**: Contributed to the large-scale training of Gecko. **Gustavo Hernandez Abrego**: Contributed to the project coordination. **Weiqiang Shi**: Contributed to the multilingual version of Gecko. **Nithi Gupta**: Contributed to the MRL implementation. **Aditya Kusupati**: Contributed to the MRL implementation. **Prateek Jain**: Contributed to the MRL implementation. **Siddhartha Reddy Jonnalagadda** Contributed to the project coordination. **Ming-Wei Chang**: Contributed to the project coordination and paper writing. **Iftekhar Naim**: Contributed to the project coordination and paper writing.

Acknowledgements

We thank Devendra Singh Sachan, Michael Kwong, Slav Petrov, and other internal reviewers from Google for reviewing our paper. We also thank Umangi Jain for the preliminary experiments on MRL.

Appendix

A. Enhancing Few-shot LLM Ranking with Ensembling

To validate the quality of the few-shot reranking, we retrieve the top 100 candidate documents and rerank them using our few-shot LLM reranker. We compare the performance of two LLM rerankers introduced in §3.2: query likelihood (QL) and relevance classification (RC). Additionally, we investigate the ensemble of these rerankers using Reciprocal Rank Fusion (RRF): $R(q, p) = 1/r_{\text{QL}}(q, p) + 1/r_{\text{RC}}(q, p)$, where $r_{\text{QL}}(q, p) > 0$ and $r_{\text{RC}}(q, p) > 0$ represent the rank positions assigned to passage p by QL and RC models for query q, respectively. It is important to note that we employ the identical prompts $\mathbb{P}_{\text{QL}}$ and $\mathbb{P}_{\text{RC}}$ used in §3.2, but not a task-specific prompt for each BEIR task.

	CV	NF	TO	DB	SF	CF	HQ	FQ	SD	FE	AR	QU	NQ	**Avg.**
Trained on MS-MARCO														
RankLLAMA	85.2	30.3	40.1	48.3	73.2	28.0	75.3	46.5	17.8	83.9	56.0	85.0	66.3	56.6
Our Few-shot Prompted LLM Re-Rankers														
Baseline	72.7	38.1	21.3	39.7	71.7	23.6	64.4	49.0	16.4	81.6	51.1	85.3	51.7	51.3
[+] QL	78.8	40.9	21.3	43.8	75.2	15.2	76.1	57.1	22.1	76.6	35.7	86.3	57.3	52.8
[+] RC	83.7	40.6	21.9	45.3	74.2	24.8	62.3	46.8	20.3	71.1	59.9	85.0	66.9	54.1
[+] RRF(QL, RC)	84.1	41.9	22.9	46.8	76.8	22.0	76.0	56.7	22.7	78.8	55.6	87.2	66.5	**56.8**

Table 6 | Few-shot LLM re-ranking performance on BEIR. We use the standard nDCG@10 metric. We report results from RankLLAMA (Ma et al., 2023), a state-of-the-art re-ranker trained on MS-MARCO, for comparison. Red indicates that the re-ranker is worse than the baseline retriever.

Table 6 shows the results. Reranking with either QL or RC improves the performance. Ensembling (RRF) significantly improves the overall quality. Importantly, the ensembled reranker consistently improves the initial retriever across all tasks except for FEVER (FE), which highlights its robustness to different tasks. This is important for creating the FRet dataset since we need high quality retrieval data across a diverse range of tasks.

B. Formatting in FRet

Since we aggregate multiple datasets from different tasks, we preprocess every input and target with a unified encoding format. In Table 7, we show that the performance of asymmetric tasks (i.e. BEIR) is sensitive to the format while the performance of symmetric tasks are relatively stable.

Symmetric Formatting	
Input	`task: {task} | query: {input}`
Target	`task: {task} | query: {target}`
Asymmetric Formatting	
Input	`task: {task} | query: {input}`
Target	`title: {title} | text: {target}`

Formatting	BEIR	STS
Input = `{task} {input}` Target = `{title} {target}`	54.7	84.8
Input = `The task is {task}, and the query is {input}` Target = `The title is {title}, and the text {target}`	54.5	**85.0**
Input = `task: {task} | query: {input}` Target = `title: {title} | text: {target}`	**55.5**	84.9

Table 7 | Formatting for FRet and other mixture datasets. We standardize different datasets and tasks in a unified encoding format (left). We also show the performance on BEIR (asymmetric formatting) and STS (symmetric formatting) with different formats (right).

C. Full MTEB Results and Instructions

In Table 8, we show the full MTEB results. In Table 9, we show the task strings (or instructions) used in the MTEB evaluation. Note that we use consistent instructions for most tasks except for BEIR, which contains multiple different intents as described in Dai et al. (2022).

	Dataset	Gecko-1B-256	Gecko-1B-768
Classification	AmazonCounterfactualClassification	70.93	75.34
	AmazonPolarityClassification	97.34	97.34
	AmazonReviewsClassification	48.47	51.17
	Banking77Classification	86.01	88.62
	EmotionClassification	51.53	52.51
	ImdbClassification	95.07	95.65
	MTOPDomainClassification	98.02	98.35
	MTOPIntentClassification	77.82	83.43
	MassiveIntentClassification	75.67	80. 22
	MassiveScenarioClassification	85.16	87.19
	ToxicConversationsClassification	88.33	89.67
	TweetSentimentExtractionClassification	72.97	74.52
Classification Pair	SprintDuplicateQuestions	96.49	96.26
	TwitterSemEval2015	78.23	79.04
	TwitterURLCorpus	87.04	87.53
STS	BIOSSES	89.42	89.46
	SICK-R	81.67	81.92
	STS12	78.02	77.59
	STS13	90.10	90.36
	STS14	85.44	85.25
	STS15	89.64	89.66
	STS16	87.24	87.34
	STS17	90.46	92.06
	STS22	67.99	68.02
	STSBenchmark	89.33	88.99
Clustering	ArxivClusteringP2P	44.12	46.27
	ArxivClusteringS2S	36.54	38.36
	BiorxivClusteringP2P	36.28	37.87
	BiorxivClusteringS2S	33.09	35.67
	MedrxivClusteringP2P	32.08	33.11
	MedrxivClusteringS2S	30.84	31.54
	RedditClustering	62.24	65.81
	RedditClusteringP2P	63.70	66.62
	StackExchangeClustering	70.19	74.52
	StackExchangeClusteringP2P	36.10	37.63
	TwentyNewsgroupsClustering	50.60	54.87
Reranking	AskUbuntuDupQuestions	63.84	64.40
	MindSmallReranking	31.89	33.07
	SciDocsRR	81.62	83.59
	StackOverflowDupQuestions	53.76	54.56
Retrieval	ArguAna	56.27	62.18
	ClimateFEVER	29.35	33.21
	CQADupstackAndroidRetrieval	45.44	48.82
	DBPedia	41.91	47.12
	FEVER	82.61	86.96
	FiQA2018	55.54	59.24
	HotpotQA	64.65	71.33
	MSMARCO	31.12	32.58
	NFCorpus	37.81	40.33
	NQ	57.37	61.28
	QuoraRetrieval	87.89	88.18
	SCIDOCS	18.21	20.35
	SciFact	70.86	75.42
	TRECCOVID	80.13	82.62
	Touche2020	27.40	25.86
Summarization	SummEval	32.36	32.63
Average		64.37	66.31

Table 8 | Results for each dataset in the MTEB benchmark.

	Dataset	Task Type	Symmetric
Classification	AmazonCounterfactualClassification	classification	✓
	AmazonPolarityClassification	classification	✓
	AmazonReviewsClassification	classification	✓
	Banking77Classification	classification	✓
	EmotionClassification	classification	✓
	ImdbClassification	classification	✓
	MTOPDomainClassification	classification	✓
	MTOPIntentClassification	classification	✓
	MassiveIntentClassification	classification	✓
	MassiveScenarioClassification	classification	✓
	ToxicConversationsClassification	classification	✓
	TweetSentimentExtractionClassification	classification	✓
Classification Pair	SprintDuplicateQuestions	semantic similarity	✓
	TwitterSemEval2015	semantic similarity	✓
	TwitterURLCorpus	semantic similarity	✓
STS	BIOSSES	semantic similarity	✓
	SICK-R	semantic similarity	✓
	STS12	semantic similarity	✓
	STS13	semantic similarity	✓
	STS14	semantic similarity	✓
	STS15	semantic similarity	✓
	STS16	semantic similarity	✓
	STS17	semantic similarity	✓
	STS22	semantic similarity	✓
	STSBenchmark	semantic similarity	✓
Clustering	ArxivClusteringP2P	search result	✓
	ArxivClusteringS2S	search result	✓
	BiorxivClusteringP2P	search result	✓
	BiorxivClusteringS2S	search result	✓
	MedrxivClusteringP2P	search result	✓
	MedrxivClusteringS2S	search result	✓
	RedditClustering	search result	✓
	RedditClusteringP2P	search result	✓
	StackExchangeClustering	search result	✓
	StackExchangeClusteringP2P	search result	✓
	TwentyNewsgroupsClustering	search result	✓
Reranking	AskUbuntuDupQuestions	question answering	
	MindSmallReranking	semantic similarity	
	SciDocsRR	question answering	
	StackOverflowDupQuestions	search result	
Retrieval	ArguAna	semantic similarity	
	ClimateFEVER	search result	
	CQADupstackAndroidRetrieval	question answering	
	DBPedia	question answering	
	FEVER	search result	
	FiQA2018	question answering	
	HotpotQA	search result	
	MSMARCO	question answering	
	NFCorpus	fact checking	
	NQ	question answering	
	QuoraRetrieval	search result	✓
	SCIDOCS	question answering	
	SciFact	fact checking	
	TRECCOVID	search result	
	Touche2020	question answering	
Summarization	SummEval	search result	

Table 9 | Instruction used for each dataset in the MTEB benchmark. Here, we denote a simplified task type (e.g., question answering) that summarizes each task generated by Gecko.